Human Safety Monitoring System

PPE Detection and Person Tracking — Technical Report

Arif Faishal

23 June 2026

Abstract

This report documents a computer-vision service for workplace human-safety monitoring. The system combines a fine-tuned YOLO model for Personal Protective Equipment (PPE) detection with a pretrained YOLOv11 model for person detection and multi-object tracking. It exposes a FastAPI HTTP interface offering single-frame detection (PPE or person) and full-video person tracking, where each tracked person is annotated with their PPE compliance state (helmet, vest, mask). Tracking-identity consistency is quantified through an ID Switch Rate metric, and throughput is reported as frames per second (FPS). The document covers the data, models, evaluation methodology, system architecture, the primary AI-assisted development steps, and current limitations.

1. Data

1.1 Inputs

The service operates on two media modalities supplied by the client at request time:

- **Images** (JPEG / PNG) — used by the single-frame detection endpoint. Bytes are decoded to a BGR array with OpenCV (`cv2.imdecode`).
- **Video** (MP4) — used by the person-tracking endpoint. The upload is buffered to a temporary file and read frame-by-frame with `cv2.VideoCapture`.

A representative operational sample, `datas/hardhat.mp4`, is retained for local testing alongside generated artifacts under `datas/detections/`.

1.2 PPE class taxonomy

The PPE model emits six mutually paired classes (a positive "wearing" class and its "missing" counterpart) that are mapped onto the application's `SafetyLabel` enumeration. The class index ordering matches the project reference in `scripts/tracker.py`.

Index	Model class	SafetyLabel
0	Hard Hat	`hard_hat`
1	No Hard Hat	`no_hard_hat`
2	Mask	`mask`
3	No Mask	`no_mask`
4	Safety Vest	`safety_vest`
5	No Safety Vest	`no_safety_vest`

The person model is class-agnostic with respect to PPE; only COCO class `0` (*person*) is requested at inference. Detected persons are mapped to the `person` label.

1.3 Outputs

Two artifact families are persisted to the configured output directory (`DETECTION_OUTPUT_DIR`, default `datas/detections`):

- **Detection**: an annotated image (`{type}_detection_{id}.{ext}`) plus a JSON response (`.json`) carrying every bounding box, label, and confidence.
- **Tracking**: an annotated MP4 (`track_person_{id}.mp4`) and a per-frame JSON file (`track_person_{id}.json`) containing tracked persons, their PPE booleans, the ID Switch Rate, and the average FPS.

2. Model

The system wraps two YOLO models through the Ultralytics runtime (`ultralytics>=8.4.75`). Both are loaded once during application startup and shared as a single in-memory instance for all requests.

Role	Weights	Size
PPE detection	`models/yolo_50.pt`	≈161 MB
Person detection/tracker	`models/yolo_person_tracker.pt`	≈5.6 MB

- **PPE model** (`yolo_50.pt`): a YOLO detector fine-tuned on the six-class PPE taxonomy above. Its larger footprint is consistent with a medium/large backbone trained for the construction-safety domain.
- **Person model** (`yolo_person_tracker.pt`): a pretrained YOLOv11 model (nano-scale, given its size) used both for person detection and as the tracking detector.

2.1 Loading lifecycle

Model weights are loaded inside the FastAPI `lifespan` context manager so that the cost is paid once at boot, never on the request path. A readiness guard (`is_ready()`) requires *both* models to be present before any endpoint will serve traffic; otherwise the API returns `503 Service Unavailable`. The `/ready` probe surfaces this state for orchestration.

2.2 Inference configuration

- **Confidence threshold**: a single configurable value (`PERSON_DETECTION_THRESHOLD`, default `0.3`) gates both detection and tracking, avoiding hard-coded thresholds.
- **Tracking**: ByteTrack (`bytetrack.yaml`) is used as the association algorithm. Tracking is driven frame-by-frame with `persist=True` so each model maintains its own track state across the video.

3. Evaluation

Evaluation focuses on the two operational qualities that matter for a live monitoring service: *tracking-identity stability* and *throughput*.

3.1 Tracking-ID consistency — ID Switch Rate

Identity stability is measured with an ID Switch Rate. A spatial cell is defined by quantising a bounding box to a 50 px grid; whenever the track ID occupying a previously seen cell changes between observations, an identity switch is counted. The rate is

$$\text{ID Switch Rate} = \frac{\#\,\text{ID switches}}{\#\,\text{frames}} \times 100\%.$$

This is reported per tracking run in the JSON output, e.g. `{"id_switches": 0, "total_frames": 25, "rate"`. The method is a lightweight, ground-truth-free proxy for identity continuity (in the spirit of MOT identity metrics) rather than a formal IDF1/MOTA computation.

3.2 Throughput — FPS

Per-frame processing latency is measured and converted to an instantaneous FPS value, which is overlaid on the top-left of every annotated frame. The average FPS over the clip is reported in the summary. In a CPU smoke test on 640 × 360 clips, the end-to-end pipeline (two YOLO models plus ByteTrack per frame) ran at roughly **9 FPS**.

3.3 Functional verification

Each module was verified end-to-end via FastAPI's `TestClient`:

- Both models load on boot and `/ready` reports ready.
- `/detect` returns `200` for `ppe` and `person`, and `422` for an invalid `detection_type`.
- `/track-person` returns `200`; track IDs, per-person PPE booleans, the ID Switch Rate, FPS, and both output files are produced (17 person-detections across a 25-frame test segment).

3.4 Not yet measured

No labelled ground truth is wired in, so detection quality (precision/recall/mAP) and formal tracking accuracy (MOTA/IDF1) are not reported. These require an annotated evaluation set and are noted in Limitations.

4. System Design

The application follows a layered FastAPI architecture with strict separation between HTTP concerns and model logic.

```
app/
  main.py                 # App factory + lifespan (loads both models on boot)
  core/
    config.py             # pydantic-settings (paths, thresholds, output dir)
    logging.py            # Logging setup
  api/
    deps.py               # Shared singletons (get_detector, get_monitor)
    v1/
      router.py           # Aggregates all v1 routers
      endpoints/
        health.py         # /health, /ready
        detections.py     # /detections (legacy single-frame)
        monitoring.py     # /monitoring/evaluate, /monitoring/alerts
        detect.py         # /detect  (detection_type: ppe | person)
        track.py          # /track-person (video -> annotated video + JSON)
```

```
  schemas/                # Pydantic DTOs (detection, monitoring, tracking)
  services/
    detector.py           # YOLO wrapper: load, detect, annotate, track_persons
    safety_monitor.py     # Detections -> events -> alerts (framework-agnostic)
  utils/
    storage.py            # Persistence helpers (image/JSON/video artifacts)
```

4.1 Layers

- **API layer** (`api/v1/endpoints`): request validation, status codes, and response models. No model code lives here.
- **Service layer** (`services/detector.py`): owns both models privately and exposes a stable interface — `detect()`, `annotate()`, and `track_persons()`. Free of FastAPI imports.
- **Schema layer** (`schemas/`): typed DTOs returned by every endpoint (e.g. `PersonDetectionResult`, `PersonTrackingResult`, `TrackingData`).
- **Utility layer** (`utils/storage.py`): file-naming templates and artifact persistence, decoupled from the request handlers.
- **Core** (`core/`): centralised configuration and logging.

4.2 Person-tracking pipeline

For each frame of the uploaded video:

1. Run the person model with ByteTrack (`classes=[0]`, `persist=True`) to obtain tracked person boxes and IDs.
2. Run the PPE model with ByteTrack to obtain PPE class boxes.
3. Associate PPE to a person when a PPE box *centre* falls inside the person box, deriving the booleans `helmet`, `vest`, `mask`.
4. Draw each person box with the meta label `ID{n} H:{0/1} V:{0/1} M:{0/1}` (green if fully compliant, red otherwise) and overlay the current FPS.
5. Update the ID Switch Rate accumulator and append the frame record.

The annotated frames are written to an MP4 (`mp4v`), and the per-frame records plus summary metrics are serialised to JSON.

4.3 Request example

```
curl -X POST http://localhost:8000/api/v1/track-person \
  -F "file=@datas/hardhat.mp4"
```

5. AI Usage Log (Primary)

The system was built incrementally with AI assistance; every session is recorded in `changelogs/agent-changelo`
The five *primary* feature-delivering steps are summarised below.

#	Prompt (paraphrased)	Outcome
3	Add a boot-time model loader for the PPE and person weights.	Added `PPE_MODEL_PATH` / `PERSON_MODEL_PATH` settings; loads both YOLO models in the lifespan; `is_ready()` now requires both.
4	Add a `detect-person` endpoint (image upload, threshold 0.3).	Implemented person detection (COCO class 0) returning `PersonDetectionResult`; added `PERSON_DETECTION_THRESHOLD` and `DETECTION_OUTPUT_DIR`.
5	Add the PPE detection module and align labels to the class taxonomy.	Replaced placeholder labels with the six PPE classes; added the class-index mapping and the `detect-ppe` path.
6	Unify the two endpoints and save annotated artifacts.	Merged into a single `/detect` endpoint keyed by `detection_type`; added OpenCV annotation and paired image + JSON persistence.
8	Build the person-tracking module.	Added ByteTrack person + PPE tracking, the ID Switch Rate, FPS overlay, and the `/track-person` endpoint producing an annotated video and tracking JSON.

6. Limitations

- **Synchronous video processing.** Tracking runs inline within the HTTP request, holding the connection open for the duration of the video. Long clips should be moved to a background job with a poll/callback contract.
- **CPU throughput.** At $\approx$9 FPS on CPU for 640×360 input, the pipeline is not real-time; GPU execution or a smaller PPE backbone would be needed for live streams.
- **Approximate PPE–person association.** Attribution uses a simple "centre-in-box" test, which can mis-assign PPE when people overlap heavily. IoU-based or nearest-person matching would be more robust.
- **Heuristic identity metric.** The ID Switch Rate uses a 50 px spatial-cell proxy, not a ground-truth alignment; it is indicative rather than a formal MOTA/IDF1 score.
- **No labelled evaluation.** Detection accuracy (precision, recall, mAP) and tracking accuracy are not yet measured against annotated data.
- **Conservative compliance semantics.** A PPE item is reported `true` only when its positive class is detected near the person; missed detections are therefore treated as non-compliant, which can inflate false "violation" signals.
- **Independent trackers.** The person and PPE models maintain separate ByteTrack states, so PPE identities are not themselves linked to a person identity over time — only re-associated per frame.
- **Output encoding.** Annotated video is written with the `mp4v` codec and no audio; container/codec compatibility is not configurable.